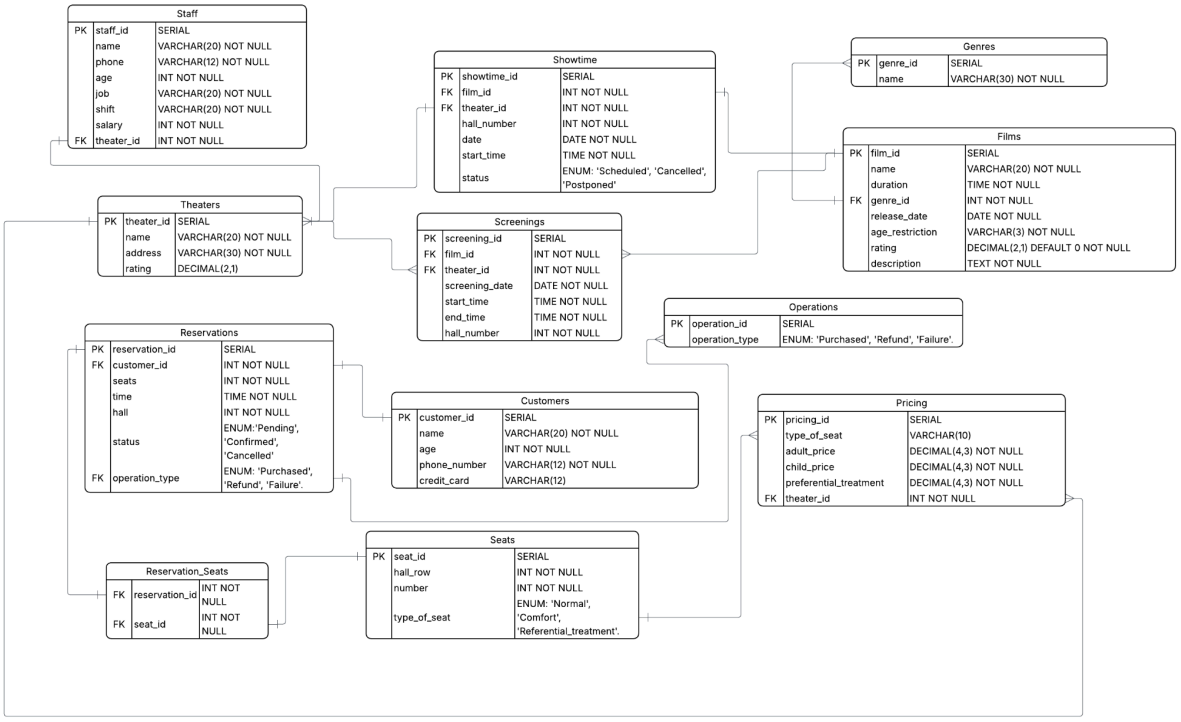
Movie Theater Management System - Database Model

This document describes the logical data model for Movie Theater Management System. It covers all entities, their attributes, inter-table relationships, sample data, and a 3NF compliance analysis.

1.1 Logical Scheme



1.2 Objects

Table: Theaters

Table Description

The Theatres table stores information about each cinema venue in the system, including its official name, physical address, and average visitor rating. Each theater is uniquely identified by theater_id (SERIAL PRIMARY KEY). This table is the central anchor for staff assignments, showtimes, screenings, and pricing.

Table Name	Field name	Field Description	Data Type / Constraints

Theaters	theater_id	PK, auto-increment	SERIAL
	name	Official name of the theater	VARCHAR(20) NOT NULL
	address	Physical location, must be UNIQUE	VARCHAR(20) NOT NULL
	rating	Average visitor rating	DECIMAL(2,1)

Comments on table relationships

One Theater has many Staff(Staff.theater_id → Theaters.theater_id). One Theater has many Showtimes(Showtime.theater_id → Theaters.theatter_id). One Theater has many Screenings(Screeings.theater_id → Theaters.theatter_id). One Theater has many Pricing records (Pricing.theater_id → Theaters.theatter_id).

Example with data

theater_id	name	address	rating
1	Chaplin	Kazakhstan, Astana, Mega 2, 2nd floor	4.8
2	Kinopark 11	Kazakhstan, Almaty, Dostyk Plaza	4.6

3	Silver Screen	Kazakhstan, Astana, Khan Shatyr	4.7
4	Lumiere	Kazakhstan, Almaty, Esentai Mall	4.9
5	CineStar	Kazakhstan, Shymkent, Mega Silk Way	4.3
6	Imax Plus	Kazakhstan, Astana, Park Mall	4.5
7	CineWorld	Kazakhstan, Karaganda, Galeria	4.2
8	Aurora	Kazakhstan, Aktau, Mega Center	4.4

Table: Staff

Table Description

The Staff table records every employee working at a theater. Each record stores the employee's full name, contact phone number, age, job, title, work shift, salary, and the theater they are assigned to. The theater_id foreign key links each employee one theater.

Table Name	Field name	Field Description	Data Type / Constraints
Staff	staff_id	PK, auto-increment	SERIAL

	name	Full name of the employee	VARCHAR(50) NOT NULL
	phone	Contact phone number UNIQUE	VARCHAR(12) NOT NULL
	age	Employee’s age in years	INT NOT NULL
	job	Job title\role	VARCHAR(50) NOT NULL
	shift	Working shift (Morning, Evening, Night)	VARCHAR(20) NOT NULL
	salary	Monthly salary in KZT	INT NOT NULL
	theater_id	FK to Theaters	INT NOT NULL

Comments on table relationships

Many Staff belong to one Theater(Staff.theater_id → Theaters.theater_id). This is a many-to-one relationship theater can employ many staff members, but each staff member is assigned to one theater at a time.

Example with data

staff_id	name	phone number	age	job	shift	salary	theater_id
1	Aibek Seitkali	+77011234567	28	Casiher	Morning	150000	1
2	Zarina Bekova	+77011234568	34	Manager	Evening	280000	1
3	Daniyar Umarov	+77011234569	22	Usher	Night	120000	2
4	Ainur Zhaksylykbekova	+77011234570	31	Projectionist	Morning	200000	2
5	Nursultan Karimov	+77011234571	45	Security	Evening	160000	3
6	Madina Ospanova	+77011234572	27	Cashier	Morning	150000	3
7	Yerlan Abenov	+77011234573	38	Manager	Morning	280000	4
8	Saltanat Nurbayeva	+77011234574	29	Manager	Morning	120000	4

Table: Seats

Table Description

The Seats table describes every physical seat available across all halls. Each seat is identified by seat_id and characterized by its row number (hall_row), seat number within that row, and category (Normal, Comfort, or Referential_treatment). The seat type directly determines the applicable ticket price via the Pricing table. Seats are linked to reservations through the Reservation_seats junction table.

Table Name	Field name	Field Description	Data Type / Constraints
Seats	seat_id	PK, auto-increment	Serial
	hall_row	Row number within the hall	INT NOT NULL
	number	Seat number within the row	INT NOT NULL
	type_of_seat	Seat category	ENUM: 'Normal', 'Comfort', 'Referential_treatment'.

Comments on table relationships

Many Seats are linked to many Reservations through the Reservation_seats junction table (Reservation_seats.saet_id → Seats.seat_id). The type_of_seat field corresponds to the same ENUM values used in Pricing.type_of_seat, enabling price look-up.

Example with data

seat_id	hall_row	number	type_of_seat

1	1	1	Normal
2	1	2	Normal
3	1	3	Normal
4	2	1	Comfort
5	2	2	Comfort
6	2	3	Comfort
7	3	1	Referential_treatmant
8	3	2	Referential_treatmant

Table: Customers

Table Description

The Customers table stores personal details about registered cinema-goers. Each customer record contains their full name, age, phone number, and credit number(stored for payment processing). The customer_id primary key uniquely identifies each registered user. Customers are referenced by the Reservations table when a booking is created.

Table Name	Field name	Field Description	Data Type / Constraints

Customers	customer_id	PK, auto-increment	Serial
	name	Full name of the customer	VARCHAR(50)
	age	Customer's age	INT
	phone	Phone number	VARCHAR(12)
	credit_card	Card number used for payments	VARCHAR(12)

Comments on table relationships

One customer can make many reservations (Reservations.customer_id → Customers.customer_id). This is a one-to-many relationship: a single customer may have multiple booking records over time.

Example with data

customeer_id	name	age	phone	credit_card
1	Aman Belov	25	+7028049823	440023409899
2	Dina Suleimanova	32	+7028049826	440023409832

3	Marat Dxhaksylykov	28	+7028049825	440023409815
4	Aigerim Nurlanova	21	+7028049823	440023409814
5	Kamila Ospanova	29	+7028049129	440023409813
6	Ruslan Berdibayev	42	+7028049912	440023409812
7	Assel Namazalieva	17	+7028049822	440023409817
8	Nurdana Khalelkyzy	18	+7028049821	440023409821

Table: Genres

Table Description

The Genres table is a reference table that defines the categories used to classify films. Each genre has a unique name. The genre_id referenced as a foreign key in the Films table, ensuring genre data is stored in one place.

Table Name	Field name	Field Description	Data Type / Constraints

Genres	genre_id	PK, auto-increment	Serial
	name	Name of the film genre	VARCHAR(50) NOT NULL

Comments on table relationships

One Genre is associated with many Films(Films.genre_id → Genres.genre_id). This is a one-to-many relationship: each film belongs to one genre, but a genre can classify many films.

Example with data

genre_id	name
1	Action
2	Comedy
3	Drama
4	Thriller
5	Animation

Table: Films

Table Description

The Films table stores the catalogue of movies available for screening. Each record includes the title, runtime in minutes, genre reference, release date, minimum age restriction, average critic/audience rating, and a short plot description. The genre_id FK links each film to its genre. Films are referenced by both the Showtime and Screening tables.

Table Name	Field name	Field Description	Data Type / Constraints
Films	film_id	PK, auto-increment	Serial
	name	Film title	VARCHAR(50)
	duration	Runtime in minutes	INT NOT NULL
	genre_id	FK to Genres	INT NOT NULL
	release_date	Official release date	DATE
	age_resctriction	Minimum viewer age (e.g. 12+)	VARCHAR(5)
	rating	Average rating	DECIMAL(2,1)
	description	Short plot synopsis	TEXT

Comments on table relationships

One Film to one Genre(Films.genre_id → Genres.genre_id). One Film can have many Showtimes and Screenings.

Example with data

film_id	name	duration	genre_id	release_date	age_restriction	rating
1	The Lion King	118	8	2019-07-21	0+	4.7
2	Avengers: Endgame	181	1	2019-04-25	12+	4.3
3	The Dark Knight	152	7	2008-07-18	16+	4.6

Table: Showtime

Table Description

The Showtime table represents the planned schedule of film screenings across theaters. Each record defines which film is screened, at which theater and hall, on what date and start time, and whether the showing is Scheduled, Cancelled, or Postponed. This table drives the public-facing timetable displayed to customers.

Table Name	Field name	Field Description	Data Type / Constraints
Showtime	showtime_id	PK, auto-increment	Serial

	film_id	FK to Films	INT NOT NULL
	theater_id	FK to Theaters	INT NOT NULL
	hall_number	Hall number within the theater	INT NOT NULL
	date	Date of the showing	DATE
	start_time	Scheduled start date-time	TIMESTAMP NOT NULL
	status	Current status of the showtime	ENUM('Scheduled', 'Cancelled', 'Postponed')

Comments on table relationships

One Showime references one Film, Theater_id. One Showtime may correspond to one Screening record once the event has started.

Example with data

showtime_id	film_id	theater_id	hall_number	date	start_time	hall_number
1	1	1	1	2024-05-10	2024-05-10 14:00	Scheduled

2	2	1	2	2024-05-10	2024-05-10 17:30	Scheduled
3	3	2	1	2024-05-12	2024-05-12 20:00	Cancelled

Table: Screenings

Table Description

The Screenings table record the actual execution of film showing. Unlike Showtime, Screenings captures the confirmed start and end times of a showing that has taken place or is currently runnnig. It also stores the hall number used. This separation supports historical reporting and analytics independent of schedule changes.

Table Name	Field name	Field Description	Data Type / Constraints
Screenings	screenig_id	Identifier, PK	Serial
	film_id	FK to Films	INT NOT NULL
	theater_id	FK to Theaters	INT NOT NULL
	screening_date	Date of the screenig	DATE NOT NULL
	start_time	Actual start time	TIMESTAMP NOT NULL

	end_time	Actual end time	TIMESTAMP NOT NULL
	hall_number	Hall used for the screening	INT NOT NULL

Comments on table relationships

One Screening references one Film, Theater. Screenings complement Showtime records: a Showtime with status ‘Scheduled’ may generate a Screening entry once the event begins.

Example with data

screenig_id	film_id	theater_id	screenig_date	start_time	end-time	hall_number
1	1	1	2024-05-10	2024-05-10 14:00	2024-05-10 16:46	1
2	2	2	2024-05-10	2024-05-10 17:30	2024-05-10 19:30	2
3	3	3	2024-05-10	2024-05-10 20:00	2024-05-10 22:28	1

Table: Reservations

Table Description

The Reservations table tracks every booking timestamp, which hall is involved, the current status, and the payment operatiom type. The opearion_id field mirrors the Operation table, for strict normalization it should be replaced with a FK to Operations.

Table Name	Field name	Field Description	Data Type / Constraints
Reservations	reservation_id	Identifier, PK	Serial
	customer_id	FK to Customers	INT NOT NULL
	seats	Number of seats reserved	INT NOT NULL
	time	Date-time when the reservation was created	TIMESTAMP NOT NULL
	hall	Hall number for the reservation	INT NOT NULL
	status	Reservation status	ENUM(‘Pending’, ‘Confirmed’, ‘Cancelled’)
	opeartion_type	Payment operation resultt	ENUM(‘Purchased’, ‘Refund’, ‘Failure’)

Comments on table relationships

Many reservations belong to one customer. One reservation can include many seats.

Example with data

reservation_id	customer_id	seats	time	hall	status	opearion_type
1	1	2	2024-05-10 13:30	1	Confirmed	Purchased
2	2	3	2024-05-10 17:00	2	Confirmed	Purchased
3	3	1	2024-05-10 09:45	2	Cancelled	Refund
4	4	4	2024-05-10 19:20	1	Pending	Failure

Table: Reservation_seats

Table Description

Reservation_seats is a bridge table that resolves the many-to-many relationship between Reservations and Seats. Each row represents one specific seat assigned to one reservation. The composite PK is (reservation_id, seat_id). This design allows a single reservation to span multiple seats and avoid repeating seat listws inside the Reservation table.

Table Name	Field name	Field Description	Data Type / Constraints
Reservation_seats	reservation_id	FK to Reservations, part of composite PK	INT NOT NULL
	seat_id	FK to Seats, part of composite PK	INT NOT NULL

Comments on table relationships

Each row referenced one reservation and one seat. Both columns form the composite PK, preventing duplicate seat assigned within the same reservation.

Example with data

reservation_id	seat_id
1	4
1	5
2	1

Table: Operations

Table Description

The Operations table is a reference table defining the three possible payment operation results Purchased, Refund, and Failure. By isolating these values in their own table, the system avoids scattering ENUM definitions across multiple tables and allows future addition of new operation types without schema changes.

Table Name	Field name	Field Description	Data Type / Constraints
Operations	operation_id	Identifier, PK	Serial
	operation_type	Label for opeartion	VARCHAR(20) NOT NULL

Comments on table relationships

One operation record can be referenced by many Reservations.

Example with data

operation_id	operation_type
1	Purchased
2	Refund
3	Failure

Table: Pricing

Table Description

The Pricing table defines the ticket price matrix for each theater, broken down by seat type and customer. Each row represents a one-seat-type price tier in one theater.

Table Name	Field name	Field Description	Data Type / Constraints
Pricing	pricing_id	Identifier, PK	Serial
	type_of_seat	Seat category	ENUM: 'Normal', 'Comfort', 'Referential_treatment'.
	adult_price	Full adult ticket price in KZT	DECIMAL(8, 2) NOT NULL
	child_price	Child ticket price in KZT	DECIMAL(8, 2) NOT NULL
	theater_id	FK	INT NOT NULL

Comments on table relationships

Each pricing record references one theater. The type_of_seats values match Seats.type_of_seat, allowing the system to look up the correct price when a reservation is confirmed.

Example with data

pricing_id	type_of_seat	adult_price	child_price	theater_id
1	Normal	1500.00	800.00	1
2	Comfort	2000.00	1700.00	2
3	Referential_treatment	1200.00	600.00	3
4	Comfort	2000.00	1700.00	4